



YACHAY TECH UNIVERSITY

SCHOOL OF MATHEMATICAL AND COMPUTATIONAL SCIENCES

PROJECT 1 – TECHNICAL REPORT

Secure Wireless Messaging for IoT Devices

DHE-PSK key exchange and Encrypt-then-MAC on two ESP32 boards

AUTHOR Daniel Troya, Jose Reinoso

COURSE Cryptography

INSTRUCTOR PhD. Christian Iza

URCUQUÍ, ECUADOR

October 2026

Contents

1	Introduction	1
2	Objectives	1
2.1	General Objective	1
2.2	Specific Objectives	1
3	Background	1
3.1	Authenticated Diffie–Hellman	2
3.2	HMAC, HKDF, and Encrypt-then-MAC	2
4	Methodology	2
4.1	Tools and Technologies	2
4.2	Procedure	3
5	Development	4
5.1	System Architecture	4
5.2	Session Key Establishment	4
5.3	Device Authentication	5
5.4	Confidentiality and Message Integrity	6
5.5	Message Format	6
5.6	Receiver Verification and Replay Protection	6
5.7	Implementation	7
5.7.1	Example Code	7
5.8	Summary of Security Requirements	7
6	Results	8
6.1	Experimental Results	8
6.2	Security Tests	9
6.3	Discussion	10
7	Key Questions and Design Insights	10
8	Conclusions	11
9	Future Work	12

References	12
A Additional Material	12
A.1 Screenshots of the Security Tests	12

1 Introduction

Small IoT devices often send data over wireless networks that anyone nearby can listen to. As a result, an attacker can read the packets, change them, inject new ones, or send old packets again. For this reason, the wireless channel must be treated as untrusted, and the security must rely on the cryptographic protocol and not on the medium. Such a system must provide confidentiality, integrity, device authentication, and replay protection at the same time, because encryption alone is not secure communication.

Our solution uses two ESP32 boards running MicroPython, called Alice and Bob, which talk over Wi-Fi with UDP. First, the boards run an ephemeral Diffie–Hellman handshake on the standard `ffdhe2048` group, authenticated with a pre-shared key (PSK), and derive the session keys with HKDF. Then every message is encrypted with AES-256-CBC and protected with an HMAC-SHA256 tag in the Encrypt-then-MAC order. Finally, a session identifier and a sequence number give replay protection. A PC on the same network acts only as the attacker and as an unauthorized device.

2 Objectives

2.1 General Objective

Design and implement a secure messaging system between two ESP32 devices that provides confidentiality, integrity, device authentication, and replay protection in a single protocol, using only standard algorithms and libraries.

2.2 Specific Objectives

- Establish a fresh, authenticated session key on every connection.
- Encrypt every message and protect it with a tag that covers the ciphertext and the header.
- Define and document a packet format with sender, session, and freshness information.
- Evaluate the prototype with the attack scenarios.
- Measure the sizes, the cryptographic times, and the latency on the ESP32.

3 Background

We assume that the attacker controls the network, so it can read, modify, drop, inject, and replay packets [1]. However, it cannot break AES, SHA-256, or the discrete logarithm problem, and it does not know the PSK stored in the authorized boards.

3.1 Authenticated Diffie–Hellman

In the Diffie–Hellman (DH) protocol, Alice sends $Y_A = g^a \bmod p$ and Bob sends $Y_B = g^b \bmod p$, and both compute the same secret

$$Z = Y_B^a \bmod p = Y_A^b \bmod p = g^{ab} \bmod p, \quad (1)$$

which an eavesdropper cannot obtain [1]. When a and b are new in every session and are erased after use (DHE), the keys of past sessions stay secret even if a long-term key is stolen later, which is known as forward secrecy. However, plain DH does not authenticate the parties, so an attacker in the middle can replace Y_A and Y_B with its own values. For this reason, the exchange must be authenticated, in our case with a PSK. In addition, we use the `ffdhe2048` group of RFC 7919, a 2048-bit safe prime with $g = 2$, and we check that $1 < Y < p - 1$ as that document requires [2].

3.2 HMAC, HKDF, and Encrypt-then-MAC

HMAC turns a hash function such as SHA-256 into a message authentication code, so only the holder of the key can produce a valid tag. HKDF uses HMAC to turn a secret that is not uniform, such as Z , into several independent keys [3]. It first extracts a pseudorandom key PRK and then expands it with a different label for every key.

AES in CBC mode gives confidentiality but not integrity. For example, if an attacker flips one bit of a ciphertext block, the same bit flips in the next plaintext block, and AES reports no error. Authenticated encryption modes such as AES-GCM solve this in one construction. When such a mode is not available, the generic composition that is proven secure is Encrypt-then-MAC: the sender computes $C = \text{Enc}_{K_{\text{enc}}}(m)$ and $T = \text{Mac}_{K_{\text{mac}}}(C)$ with independent keys, and the receiver checks T before decrypting [1]. As a result, a modified ciphertext is never decrypted, which also prevents padding oracle attacks.

4 Methodology

4.1 Tools and Technologies

Table 1 lists the tools that we used. All the cryptographic primitives come from standard libraries.

Table 1. Tools and technologies used in this work.

Tool	Purpose
$2 \times$ ESP32 + MicroPython	Alice and Bob. MicroPython provides <code>cryptolib</code> (AES), <code>hashlib</code> , <code>hmac</code> , <code>os.urandom</code> , and <code>pow</code> .
Python 3 on a PC	Attacker, unauthorized device, and plots (<code>cryptography</code> , <code>matplotlib</code>).
<code>esptool</code> , <code>mpremote</code>	Flashing the firmware, uploading the code, and opening the console.
Phone hotspot (2.4 GHz)	Wi-Fi network shared by the boards and the PC.
Git and GitHub	Version control and code sharing.

4.2 Procedure

The system was built incrementally. First, each cryptographic piece was written as a small module; then the modules were joined into the protocol; finally, the protocol was connected to the network code of the boards. Each module includes a self-test that compares it with official vectors (NIST SP 800-38A for AES-CBC, RFC 4231 for HMAC, and RFC 5869 for HKDF). Moreover, the same modules run unchanged on the PC and on the ESP32.

In the test bench, Alice (ID 0x01) starts the handshake and Bob (ID 0x02) waits for it, both on UDP port 5005. For the attacks, the boards are uploaded with the option `-Atacante`, so they send every packet to the PC on port 6000, and `attacker.py` forwards it to the other board after applying the selected attack. The attacker does not know the PSK. Table 2 shows the test plan and the behavior that the design expects.

Table 2. Security test plan and expected behavior of the receiver.

Test	How it is done	Expected behavior
Normal exchange	Direct, and <code>attacker.py --mode sniff</code>	Same SID on both boards; the attacker sees only public values and ciphertext.
Modified cipher-text	<code>--mode tamper_c</code> flips one bit of C	Rejected: TAG
Modified tag	<code>--mode tamper_tag</code> flips one bit of the tag	Rejected: TAG
Replay	<code>--mode replay</code> resends an accepted packet	Second copy: Rejected: replay
Forged packet	<code>--mode forge</code> invents IV , C , and tag	Rejected: TAG
MITM	<code>--mode mitm</code> replaces Y_A	Alice: Rejected: handshake
Unauthorized device	<code>node.py --id 0x99</code> and <code>node.py --psk-falsa</code>	Bob: Rejected: handshake

The measurements were taken only on the ESP32, because the goal is to know the cost of security on the IoT platform itself. The script `medir.py` measures *seal* (padding, encryption, and tag) and *open* (checks, decryption, and unpadding) for $L = 8, 32, 128$, and 512 bytes, with 10 warm-up runs and 100 measured runs per size. It also measures the rejection of a packet with a bad tag and the full handshake ($n = 3$). For the latency, Alice sends 50 protected PING messages and Bob answers each with a protected PONG; the one-way latency is estimated as half of the round-trip time (RTT).

5 Development

5.1 System Architecture

As figure 1 shows, both boards run the same layered code. `main.py` handles Wi-Fi, the UDP socket, the menu, and the LED; `protocol.py` contains the handshake and the `seal` and `open` functions; and the small modules below it call the standard libraries of MicroPython. Only the configuration file changes between Alice and Bob, since it sets the ID, the role, the peer address, and the PSK.

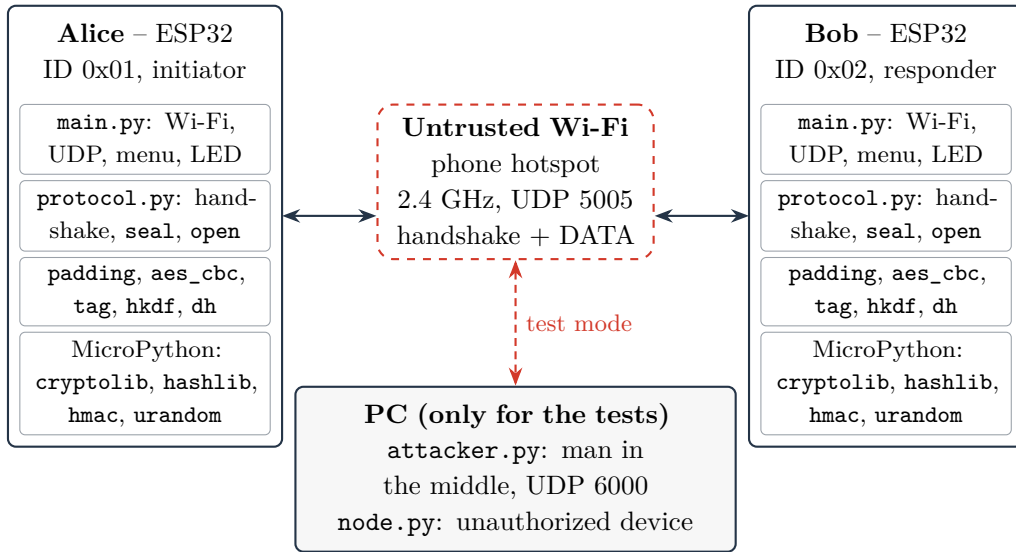


Figure 1. System architecture. The PC is used only in the security tests.

5.2 Session Key Establishment

We chose an ephemeral DH exchange authenticated with a PSK (DHE-PSK), which combines key agreement and pre-shared authentication. DH gives a new key in every session and forward secrecy, while the PSK gives authentication. Figure 2 shows the three handshake messages and the first data packet.

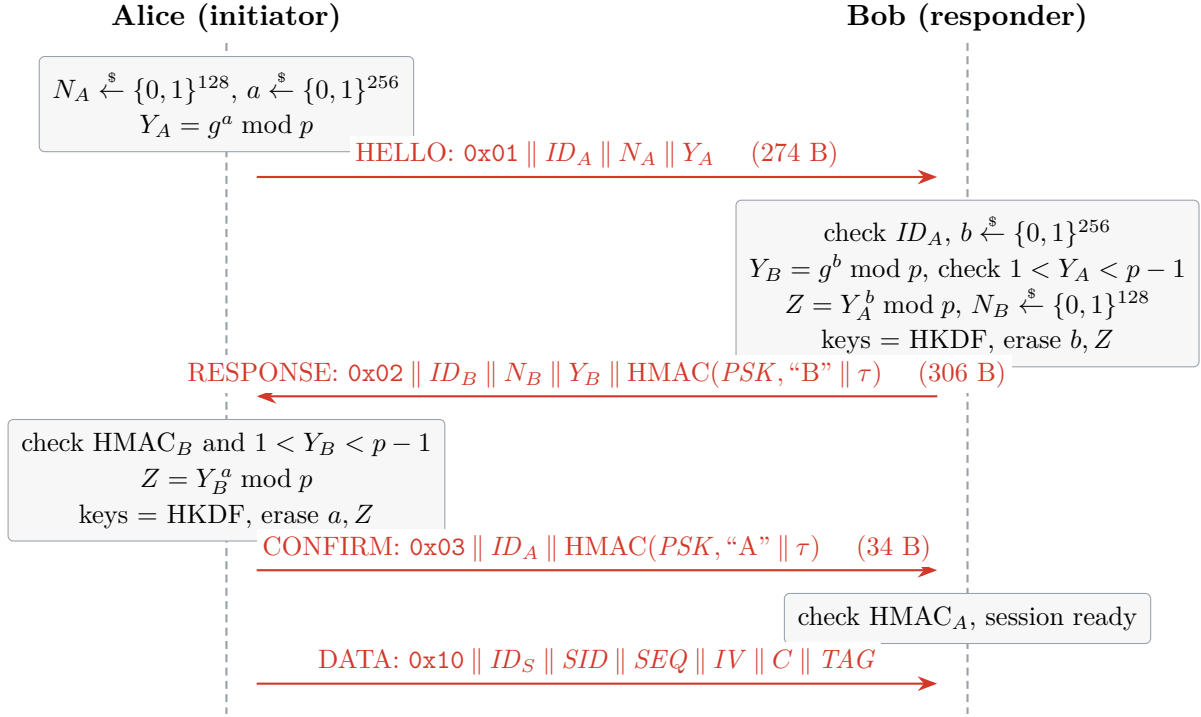


Figure 2. Protocol sequence diagram: DHE-PSK handshake and the first protected message.

Alice sends a 16-byte nonce N_A and Y_A , computed with a 256-bit exponent, which is enough for this group according to RFC 7919 [2] and much faster on the ESP32 than a 2048-bit exponent. Bob checks that the sender ID is authorized, validates Y_A , and computes Z . Both sides then use the transcript $\tau = ID_A \parallel ID_B \parallel N_A \parallel N_B \parallel Y_A \parallel Y_B$ and derive the session material with HKDF [3]:

$$\begin{aligned}
 PRK &= \text{HMAC}(N_A \parallel N_B, Z \parallel PSK), \\
 K_{\text{enc}}, K_{\text{mac}}, SID &= \text{Expand}(PRK, \text{"enc"}, \text{"mac"}, \text{"sid"}).
 \end{aligned} \tag{2}$$

The nonces act as the salt, so two sessions never share keys, and the PSK is also mixed into the key material. K_{enc} and K_{mac} are 32 bytes each and independent, as Encrypt-then-MAC requires, while the 4-byte SID is obtained by both boards without being sent.

Regarding the management of the cryptographic material, the exponents and Z are erased right after the derivation, and the session keys live only in RAM. The PSK is a random 32-byte value made with `os.urandom` and stored only in the configuration file of each board, which is excluded from GitHub with `.gitignore`.

5.3 Device Authentication

A device is authorized only if its ID is in the PSK table of the other board and it proves that it knows the PSK. Bob proves it with $\text{HMAC}(PSK, \text{"B"} \parallel \tau)$ and Alice with $\text{HMAC}(PSK, \text{"A"} \parallel \tau)$. Since these values cover Y_A and Y_B , an attacker who replaces a public value breaks the check, which stops the man-in-the-middle attack. In addition, the labels "A" and "B" prevent reflecting Bob's value back to him, and the fresh nonces prevent replaying an old handshake. Bob only creates the session after a valid CONFIRM, so a device without the PSK never gets one. We used a PSK instead of signatures because MicroPython has no signature library, and writing

RSA or ECDSA by hand would be custom cryptography.

5.4 Confidentiality and Message Integrity

Each message is padded with PKCS#7, encrypted with AES-256-CBC under K_{enc} and a new random IV , and then protected with

$$TAG = \text{trunc}_{16}(\text{HMAC}(K_{\text{mac}}, \text{TYPE} \parallel ID_S \parallel SID \parallel SEQ \parallel IV \parallel C)). \quad (3)$$

Because the tag covers the header and not only C , any change in the ciphertext, the sender, the session, or the sequence number invalidates it, and the receiver rejects the packet. The 16-byte tag leaves a forgery probability of about 2^{-128} .

Authenticated encryption modes such as AES-GCM or ChaCha20-Poly1305 are the usual choice for this task. However, `cryptolib` on the ESP32 only offers AES in ECB, CBC, and CTR modes, and writing GCM by hand would be custom cryptography. Therefore, we combined two standard primitives in the Encrypt-then-MAC order with independent keys, which is the composition proven secure [1].

5.5 Message Format

Our packet follows the common structure $ID_S \parallel SID \parallel SEQ \parallel N \parallel C \parallel TAG$ with two changes. First, a one-byte $TYPE$ separates handshake and data packets on the same socket. Second, the nonce N is the CBC IV . Table 3 describes the fields.

Table 3. Format of the protected DATA packet (L is the plaintext length).

Field	Size (B)	Purpose
$TYPE$	1	0x10 for data (0x01–0x03 for the handshake).
ID_S	1	Sender; it must be the peer of the session.
SID	4	Current session (derived by HKDF).
SEQ	4	Sequence number, from 1 and increasing; gives freshness.
IV	16	Random IV of AES-CBC (the nonce N).
C	$16(\lfloor L/16 \rfloor + 1)$	Ciphertext of the padded message.
TAG	16	Truncated HMAC-SHA256 of all the previous fields.

Hence, a protected packet has $|P| = 42 + 16(\lfloor L/16 \rfloor + 1)$ bytes. The handshake messages have fixed sizes (274, 306, and 34 bytes, that is, 614 bytes per session), and any message with a wrong length or type is rejected.

5.6 Receiver Verification and Replay Protection

A message is accepted only after its origin and integrity are verified. Listing 1 shows the order of the checks in `open`. The cheap checks of the session and the sender come first. Then the tag is verified in constant time, before any field is trusted or any byte is decrypted, which prevents padding oracle attacks. Only after that the sequence number is checked, so a forged packet

with a huge *SEQ* cannot block the real messages.

Replay protection combines a session identifier and a sequence number. Inside a session, a packet is accepted only if its *SEQ* is larger than the last accepted one, so a captured packet sent again is rejected even though its tag is valid. A packet from an old session has another *SID* and other keys, so it is rejected too. We did not use timestamps because the boards have no synchronized clock. The cost of this rule is that UDP packets arriving out of order are dropped, which is acceptable for short messages sent one at a time.

5.7 Implementation

The code is organized in `common/` (cryptography and protocol), `esp32/` (board program), and `pc/` (attacker and intruder). The `main.py` program retries the handshake after a rejection, so a failed attempt does not stop the board. Moreover, the LED blinks once for an accepted message and three times for a rejected one, and every send or receive prints the sizes and the time of `seal` or `open`. The README of the repository lists every command needed to reproduce the experiment.

5.7.1 Example Code

Listing 1. Receiver checks in `Session.open` (`common/protocol.py`).

```

1 def open(self, packet):
2     if len(packet) < 58 or (len(packet) - 42) % 16 != 0:
3         raise Rejected("format")
4     # 1) SID and ID: fast drop, without computing the HMAC
5     if packet[0] != DATA or packet[1] != self.peer_id or packet[2:6] != self.sid:
6         raise Rejected("session/ID")
7     # 2) TAG before anything else (without this: padding oracle and DoS on the
counter)
8     if not ct_equal(make_tag(self.k_mac, packet[:-TAG_LEN]), packet[-TAG_LEN:]):
9         raise Rejected("TAG")
10    # 3) SEQ, already authenticated by the TAG
11    seq = int.from_bytes(packet[6:10], "big")
12    if seq <= self.ultimo_seq:
13        raise Rejected("replay")
14    self.ultimo_seq = seq
15    # 4) Decrypt and remove padding
16    try:
17        return pkcs7_unpad(aes_cbc_decrypt(self.k_enc, packet[10:26], packet[26:-
TAG_LEN]))
18    except ValueError:
19        raise Rejected("format")

```

5.8 Summary of Security Requirements

Table 4 summarizes how the design meets each requirement.

Table 4. Security requirements and how the design meets them.

Requirement	How it is met
Session key	DHE on <code>ffdhe2048</code> , HKDF, independent keys, secrets erased (section 5.2).
Confidentiality	AES-256-CBC with a random IV per message.
Integrity	HMAC-SHA256 tag over header, IV, and C (Encrypt-then-MAC), checked first.
Device authentication	PSK per device ID and HMAC over the whole transcript.
Replay protection	SID per session and increasing SEQ .
Standard algorithms	AES, SHA-256, HMAC, HKDF, and DH from the platform libraries.
Message format	Table 3.
Security tests	<code>attacker.py</code> and <code>node.py</code> (table 2).
Evaluation	<code>medir.py</code> and the latency option, on the ESP32 (section 6).

6 Results

All times were measured on the ESP32 with `time.ticks_us()` and are reported as mean $\pm$ standard deviation.

6.1 Experimental Results

Normal communication. Alice and Bob completed the handshake over Wi-Fi and both printed `Handshake complete` with the same SID . The messages sent from one board were verified, decrypted, and shown correctly on the other. In addition, all 50 PING and 50 PONG packets of the latency test were accepted, which confirms that the whole protocol works on the real hardware.

Sizes and times. Table 5 and figure 3 show the results for each plaintext size. The packet sizes match the formula of the previous section exactly. Sealing takes about 69 ms and opening about 70 ms, almost independently of L . Moreover, rejecting a packet with a bad tag ($L = 32$) took 68.86 ± 1.30 ms, nearly the same as a full open. The full handshake, with both roles on one board, took 7626.6 ± 23.4 ms ($n = 3$) and sends 614 bytes once per session.

Table 5. Packet size and time per message on the ESP32 ($n = 100$ per size).

L (B)	Packet (B)	Overhead (B)	Overhead	Seal (ms)	Open (ms)
8	58	50	625%	68.95 ± 0.97	69.54 ± 0.85
32	90	58	181%	69.22 ± 1.34	70.10 ± 1.50
128	186	58	45%	69.30 ± 1.72	69.73 ± 1.04
512	570	58	11%	69.73 ± 1.68	70.56 ± 1.89

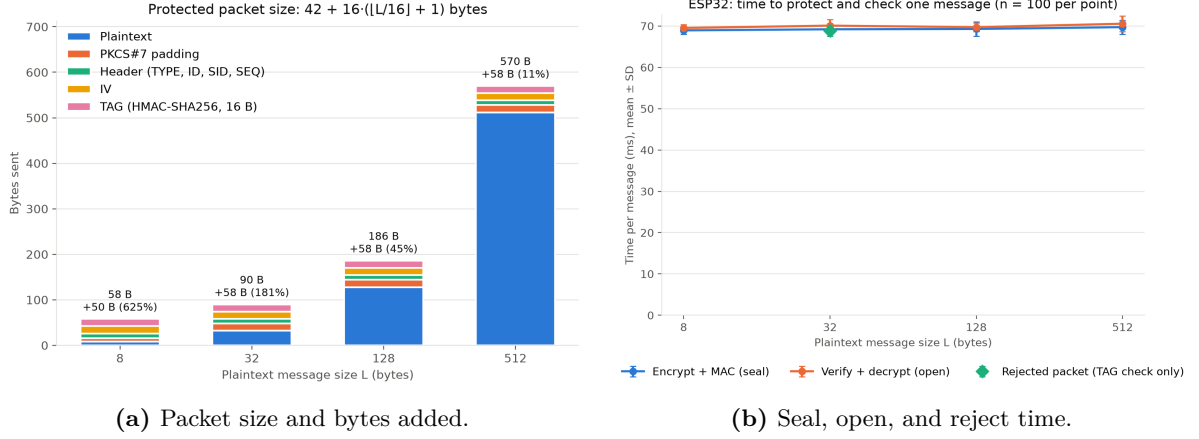


Figure 3. Cost of protecting one message on the ESP32.

Latency. The RTT of a protected PING/PONG was 494.7 ± 87.3 ms ($n = 50$, from 375.8 to 659.0 ms), so the one-way latency is about 247 ms. Each round trip includes two seal and two open operations, about $2 \times (68.95 + 69.54) \approx 277$ ms, that is, 56% of the RTT (figure 4).

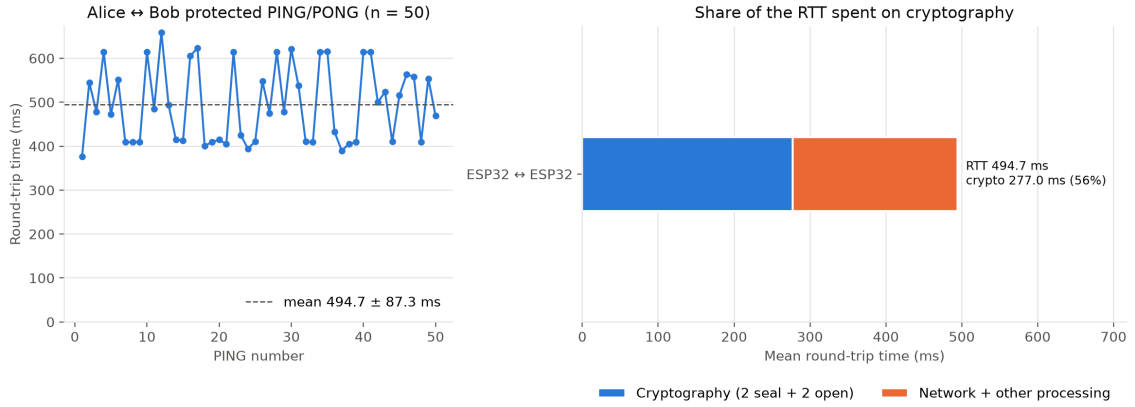


Figure 4. RTT of 50 protected PING/PONG messages between the boards, and the share spent on cryptography.

6.2 Security Tests

The laptop was placed between the boards as the attacker (`attacker.py`), so every packet from Alice to Bob went through it. The console captures are in section A.1.

Sniffing. The attacker forwarded the handshake (HELLO 274 B, RESPONSE 306 B, CONFIRM 34 B) and the data packet without changing it. Bob printed the message `Hey Bob!` correctly, while the attacker only saw the fields TYPE, IDS, SID, SEQ, IV, C, and TAG as bytes, so the text was not readable (figures 5 and 6).

Modified ciphertext. The attacker changed one bit of C (`f9` → `f8`) and left the rest of the packet as it was. Bob printed `Rejected: TAG` and did not show any message (figures 7 and 8).

Replay. The attacker sent the same valid packet twice. Bob accepted the first one and printed the message, and then printed **Rejected: replay** for the second one, which was identical (figures 9 and 10).

Forged packet. The attacker replaced IV , C , and TAG with values of its own, without knowing the keys. Bob printed **Rejected: TAG** (figures 11 and 12).

6.3 Discussion

The main cost of each message is the HMAC and not AES. Rejecting a packet only computes the HMAC and costs almost the same as a full open, and the time hardly changes from 8 to 512 bytes. A likely reason is that the `hmac` module of MicroPython is written in Python, while AES runs in native code. The handshake costs about 110 times more than sealing one message because of its modular exponentiations, but it happens only once per session.

In terms of communication, the protection adds 50 to 58 bytes per message. This is large for very short messages (625% for 8 bytes) but small for long ones (11% for 512 bytes), and every field is needed for one of the required properties. Finally, cryptography takes about 56% of the RTT, and the rest is the Wi-Fi network. The RTT varies a lot between samples, probably because of the power saving of the ESP32 Wi-Fi and the phone hotspot, while the cryptographic times vary by only 1 to 2 ms. Overall, a quarter of a second per direction is acceptable for short text messages.

The security tests agree with the order of the checks in listing 1. In the sniffing test the attacker saw the header in clear text, but C and TAG were only bytes, because the content is encrypted with a key that never travels on the network. When the attacker changed one bit of C or replaced IV , C , and TAG , the second check failed: the HMAC covers the whole header and the ciphertext, and without K_{mac} the attacker cannot compute a valid one. Since this check happens before decrypting, Bob did not process any of the altered data. The replay was different: the repeated packet was a real one, so its TAG was valid and it passed the second check, and it was the third check (`seq <= ultimo_seq`) that rejected it. This shows that the counter is needed even when the TAG is correct.

7 Key Questions and Design Insights

This section answers the most important questions about the design and points out some distinctions that are easy to miss.

Is encryption enough to secure the messages?

No. Encryption hides the content, but it does not stop an attacker from changing or repeating a packet. For example, flipping one bit of a CBC ciphertext flips the same bit in the recovered plaintext without any error. Therefore, confidentiality, integrity, authentication, and freshness must be provided together, which is why every packet carries a tag, a session identifier, and a sequence number.

Why use a PSK if Diffie–Hellman already gives a shared key?

Plain DH only guarantees that two parties share a key, not who those parties are. An attacker in the middle could run one DH exchange with each board. The PSK binds the exchange to the authorized devices, because only they can compute the HMAC over the transcript. In addition, MicroPython has no signature library, so a PSK is the practical way to authenticate the boards.

What happens if the PSK is stolen?

Past sessions remain secret, because their keys also depend on the ephemeral exponents, which were erased. However, an attacker with the PSK could impersonate a board in future sessions. Therefore, a stolen PSK must be replaced on both boards. This is the main difference between using DHE-PSK and using only a PSK, where a stolen key would also reveal every recorded session.

Why AES-CBC with HMAC instead of AES-GCM?

AES-GCM is not available in `cryptolib`, and implementing it by hand would be custom cryptography. Encrypt-then-MAC with independent keys reaches the same goal with standard primitives, as long as the tag is checked before decryption.

Why a standard group and a 256-bit exponent?

Inventing DH parameters is risky, while `ffdhe2048` is a published and reviewed group. Moreover, RFC 7919 asks for an exponent of at least 224 bits for this group, so 256 bits keeps the security level and makes each exponentiation much faster on the ESP32.

Why is a session identifier needed if the keys change in every session?

The keys alone would already reject old packets, but only after computing the HMAC. The *SID* lets the receiver drop packets from other sessions at once and report a clear reason for the rejection.

What does the system not protect?

The device IDs travel in clear, so an observer knows who is talking but not what is said. Moreover, the protocol cannot stop an attacker from blocking packets or from sending many HELLO messages to keep the board busy with the 7.6 s handshake, which are denial-of-service attacks that cryptography alone cannot prevent. Finally, a person with physical access to a board could read the PSK from its flash memory.

8 Conclusions

We built a secure messaging system between two ESP32 boards that combines confidentiality, integrity, device authentication, and replay protection in one protocol, using only standard algorithms and libraries. The session key comes from an ephemeral DH exchange authenticated with a PSK, and HKDF derives independent keys and a session identifier. Since MicroPython has no AES-GCM, we justified and used AES-256-CBC with HMAC-SHA256 in the Encrypt-then-MAC order, with a tag that covers the whole header and is checked before decryption.

The normal exchange between the two boards works, and the measurements show the practical cost on the ESP32: about 70 ms and 50 to 58 bytes per message, mostly due to the HMAC, plus a handshake of about 7.6 s once per session.

In the security tests, the laptop acted as an attacker between the boards. It could not read the message, and every modified, repeated, or forged packet was rejected by Bob, with **Rejected: TAG** for the first and the last, and **Rejected: replay** for the second. The tests of the MITM attack and of the unauthorized device were not run, so the device authentication is supported only by the design and not by an experiment.

9 Future Work

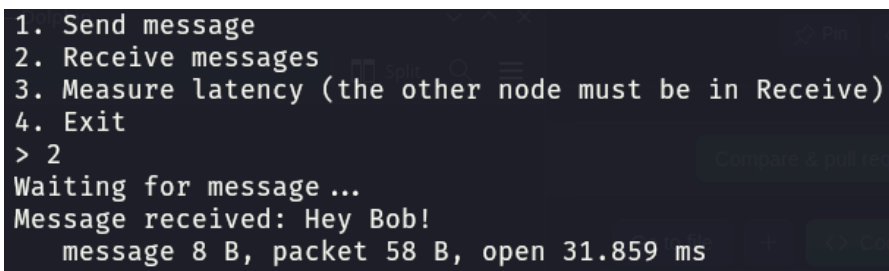
If a future firmware offers AES-GCM or ChaCha20-Poly1305, the messages could use a single authenticated encryption mode, and ECDH (for example X25519) would make the handshake faster and smaller. In addition, a sliding window for *SEQ*, a limit on HELLO messages, and the flash encryption of the ESP32 would address the limits described in the previous section.

References

- [1] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2020.
- [2] D. K. Gillmor, “Negotiated finite field Diffie-Hellman ephemeral parameters for transport layer security (TLS),” Internet Engineering Task Force, RFC 7919, 2016.
- [3] H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF),” Internet Engineering Task Force, RFC 5869, 2010.

A Additional Material

A.1 Screenshots of the Security Tests



```

1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Message received: Hey Bob!
message 8 B, packet 58 B, open 31.859 ms

```

Figure 5. Normal exchange: Bob receives the message.

```

Attacker listening on port 6000 | mode: sniff
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
HELLO 274 B | ID: 0x1
N: 4f46da8cd46cce0b63d85542e47a5b52
Y: 25078c0061007760f72adf76f193f3c3 ...
HELLO forwarded
B → A
RESPONSE 306 B | ID: 0x2
N: 6b7bad06f7d7db155694ba435e78e8d8
Y: b85ccc39ba6a7dcf84fc09872ea54136 ...
RESPONSE forwarded
A → B
CONFIRM 34 B | ID: 0x1
CONFIRM forwarded
A → B
TYPE: 0x10
IDS: 0x1
SID: 30b48fce
SEQ: 1
IV: 4da07b3c7a2c0c6a109ffbb9d9cc6df8
C: 60291be2fb471871fda5206ec2142937
TAG: dffffae3cb9c454f142801c5cbae0b03d
DATA forwarded

```

Figure 6. Sniffing: the attacker only sees bytes.

```

Attacker listening on port 6000 | mode: tamper_c
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
Original packet received:
TYPE: 0x10
IDS: 0x1
SID: 30b48fce
SEQ: 1
IV: d2dba9f71a5d27d86f781515c7279ef8
C: f922d5c2f26a981a8195c2b1d0f0359b
TAG: efd4a4376a8e2e40619963424247d058
Packet sent (ciphertext modified, 1 bit):
TYPE: 0x10
IDS: 0x1
SID: 30b48fce
SEQ: 2
IV: d2dba9f71a5d27d86f781515c7279ef8
C: f822d5c2f26a981a8195c2b1d0f0359b ← modified
TAG: efd4a4376a8e2e40619963424247d058

```

Figure 7. Modified ciphertext: the attacker changes one bit of C .

```

1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Rejected: TAG | packet 58 B

```

Figure 8. Modified ciphertext: Bob rejects the packet (TAG).

```

Attacker listening on port 6000 | mode: replay
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
Original packet received:
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 3
  IV: d15ada9bbb66ee2a5f0fc2c0ece53b2f
  C: 93bb231eab929b2d2ee40f8e4ac9d65b
  TAG: c348f3e87ffef9355714fa6970f36f54
Packet sent (1st, normal)
Packet sent again (2nd, the replay, identical to the 1st):
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 3
  IV: d15ada9bbb66ee2a5f0fc2c0ece53b2f
  C: 93bb231eab929b2d2ee40f8e4ac9d65b
  TAG: c348f3e87ffef9355714fa6970f36f54

```

Figure 9. Replay: the attacker sends the same packet twice.

```

1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Message received: Hey Bob
  message 7 B, packet 58 B, open 32.543 ms
Rejected: replay | packet 58 B

```

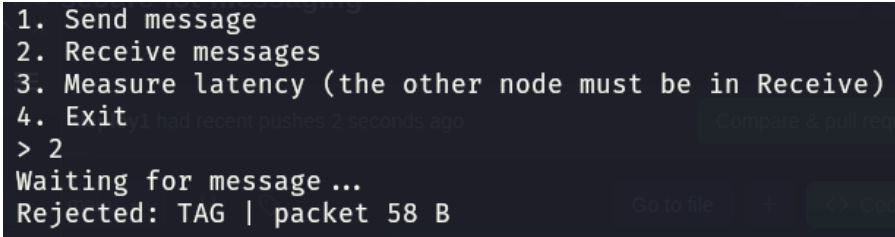
Figure 10. Replay: Bob accepts the first packet and rejects the second.

```

Attacker listening on port 6000 | mode: forge
Node A: ('10.32.7.41', 5005) | Node B: ('10.32.7.224', 5005)
A → B
Original packet received:
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 4
  IV: b445f9694484b822a55a351347fae1cb
  C: 8d5a1fb81bc6cd9b958120dcbf54ef1
  TAG: fef761bd7a39de1b05a32f77301ad5b1
Fake packet sent instead of the original:
  TYPE: 0x10
  IDS: 0x1
  SID: 30b48fce
  SEQ: 4
  IV: c3d46ef347fbcdb48fd701555c74b2f2 ← modified
  C: fd2c8b97f2f74363cf1f60c6f4a824b7 ← modified
  TAG: e7b5d89b0bda0f68166b09949e5cfbfd ← modified

```

Figure 11. Forgery: the attacker replaces IV, C, and TAG.



```
1. Send message
2. Receive messages
3. Measure latency (the other node must be in Receive)
4. Exit
> 2
Waiting for message ...
Rejected: TAG | packet 58 B
```

Figure 12. Forgery: Bob rejects the packet (TAG).